COMP3173 23F

Project Implementation

Phase 2 Syntax Analysis

In phase 1, you have already implemented a lexer for the language as a function call "`next_token`" and the lexer can store identifiers in the symbol table. In the second phase, you will implement an SLR(1) parser for the language. The parser will take a token from the lexer as its input each time and analyze the structure of the source code. If the source code does not have any syntax error, the parser will build a parse tree; otherwise, the parser reports the error.

For the features of the language, you are referred to "Project Description". The grammar of this language is listed as below.

Grammar

```
0.    S' -> S
1.    S  -> F ; S
2.    S  -> C
3.    F  -> fun id A -> C
4.    A  -> T id A
5.    A  -> T id
6.    T  -> nat
7.    T  -> bool
8.    C  -> B ? E : C
9.    C  -> E
10.   E  -> ( id I )
11.   I  -> E I
12.   I  -> E
13.   E  -> lit + E
14.   E  -> lit
15.   B  -> E R E & B
16.   B  -> E R E
17.   R  -> <
18.   R  -> =
```

To make your implementation easier, the parsing table is given in "COMP3173 23F Project Parsing Table.xlsx". You don't need to build the table by yourself.

Note that the right-hand side of Rule 3 has the token "`->`" for function declarations, which is different from the grammar production arrow.

Implementation Requirements:

To implement the parser, you need to write 4 source files: "`parser.h`", "`parser.c`", "`parse_tree.h`", and "`parse_tree.c`" and modify "`func.c`".

"`lexer.h`", "`lexer.c`", "`symbol_table.h`", and "`symbol_table.c`" do not need to be changed if there is no bug in Phase 1. These source files should be packed and submitted together within Phase 2 (otherwise, the parser will not get tokens).

"`func.c`"

- Once a token is returned by the lexer, the main function will pass the token to parser.
- The parser will try to analyze the syntax of the source program.
- The main function also initializes the parse tree, which is used to store the outcome of the parser.
- If the parser returns an error flag, stop the analysis and print out an error message on the screen.
- After the entire process is finished, show that the syntax analysis is complete on the screen.
- Use file write to output the outcome of each step of parsing into a txt file. Suppose the source code is "test1.txt", then the output file should be named "test1_out.txt". And try your best to properly align the outcome.

"`parser.h`" and "`parser.c`"

- Use a static area in "`parser.h`" to define the parsing table (like how you build the transition table in phase 1).
- For each step of the paring, the parser needs to print the current configuration in the stack and the output on the screen.
- The output is either "`shift` x" or "`reduce` R", where x is a state and R is a production rule.
- When the parser reduces, it also modifies the parse tree at the same time.
- If there is any syntax error, the parser returns an error flag to the main function.

"`parse_tree.h`" and "`pares_tree.c`"

- It is used to store the parse tree of a parser.
- Each internal vertex of the tree has one or multiple children.

The implementation must be done in **standard C** (not in Visual C). For those who do not have standard C installed on your local computer, you can try the GCC online at https://www.onlinegdb.com/ . TA will use make file to check your analyzer.

Example:

Here are two simple test cases and the expected outcomes.

“test1.txt”:

```
fun f1 nat a -> T;
F
```

“test1_out.txt”:

```
Step 0:(0,
Step 1:(0, fun 4,                                   shift 5
Step 2:(0, fun 4, id 10,                            shift 10
Step 3:(0, fun 4, id 10, nat 20,                    shift 20
Step 4:(0, fun 4, id 10, T 19,                      reduce T -> nat
Step 5:(0, fun 4, id 10, T 19, id 28,               shift 28
Step 6:(0, fun 4, id 10, A 18,                      reduce A -> T id
Step 7:(0, fun 4, id 10, A 18, -> 27,               shift 27
Step 8:(0, fun 4, id 10, A 18, -> 27, lit 8,        shift 8
Step 9:(0, fun 4, id 10, A 18, -> 27, E 6,          reduce E -> lit
Step10:(0, fun 4, id 10, A 18, -> 27, C 33,         reduce C -> E
Step11:(0, F 2,                                     reduce F -> fun id A -> C
Step12:(0, F 2, ; 9,                                shift 9
Step13:(0, F 2, ; 9, lit 8,                         shift 8
Step14:(0, F 2, ; 9, E 6,                           reduce E -> lit
Step15:(0, F 2, ; 9, C 3,                           reduce C -> E
Step16:(0, F 2, ; 9, S 17,                          reduce S -> C
Step17:(0, S 1,                                     reduce S -> F ; S
Step18:(0, S 1, $                                   accept
```

Screen message

```
Parsing Accomplished! No syntax error!
```

“test2.txt”:

```
1 < 2 ? T
```

“test2_out.txt”:

```
Step 0:⟨0,
Step 1:⟨0, lit 8,                                   shift 8
Step 2:⟨0, E 6,                                     reduce E -> lit
Step 3:⟨0, E 6, < 13,                               shift 13
Step 4:⟨0, E 6, R 12,                               reduce R -> <
Step 5:⟨0, E 6, R 12, lit 8,                        shift 8
Step 6:⟨0, E 6, R 12, E 23,                         reduce E -> lit
Step 7:⟨0, B 5,                                     reduce B -> B R E
Step 8:⟨0, B 5, ? 11,                               shift 11
Step 9:⟨0, B 5, ? 11, lit 8,                        shift 8
Step10:⟨0, B 5, ? 11, E 22,                         reduce E -> lit
Step11:⟨0, B 5, ? 11, E 22, $                       syntax error
```

Screen message:

```
Syntax error on Line 1!
```

Note: if you have difficulties on printing “underlines” on the screen, you can also use brackets to denote a pair in configurations. For example,

```
⟨0, (lit 8),
```

For more test cases you can try to write by yourself.

Submission requirements

Each team need to clearly indicate the contribution of each team member in a txt file. To submit your work, you need to pack all files (source code and contribution txt) in a package. Rename the package as COMP3173_23F_SectionXX_TeamYY, where XX is your section number and YY is your team number. Only team leaders need to upload the package to iSpace.